COMP6010 Assignment: From Bayesian Inference to Generative AI

Planning and Handling Uncertainty in Machine Learning

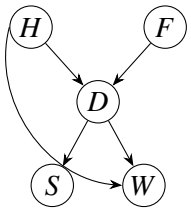
Satria Pamungkas • ID: **21522611** • Part C option: **C2** (animal drawings, flow matching)

Introduction. This report answers three questions using probability. Part A does *inference*: the model is fully known, and the job is to use evidence correctly. Part B *learns* one unknown number, and shows why a decision must use the whole posterior instead of a single best guess. Part C *learns a generative model* — a flow-matching network trained from scratch on Quick, Draw! drawings — showing the same pattern works when the unknown is a whole function. Every number comes from the supplement's notebooks and is saved in a named file, shown in `grey type`. The unit materials [1] are the primary source throughout.

Part A: Bayesian network simulation and sampling

A1. The model and exact answers

Five yes/no variables: H (hot weather), F (irrigation fault), D (dry soil), S (sensor says "dry"), W (plants wilting). Arrows are $H \to D$, $F \to D$, $D \to S$, $D \to W$, $H \to W$. The two causes at the top are independent, with $P(H{=}1) = 0.30$ and $P(F{=}1) = 0.08$.



The arrows split the joint probability into one small piece per variable:

$$P(H,F,D,S,W) = P(H)\,P(F)\,P(D \mid H,F)\,P(S \mid D)\,P(W \mid H,D).$$

That is 2+2+4+2+4 table entries instead of the 31 numbers a full joint over five yes/no variables would need — the graph is what saves that work.
One independence the graph gives is $S \perp W \mid D,H$: the sensor connects to the network only through D, its single parent, so once we know D (and H, which also feeds W) the sensor tells us nothing new about wilting. Checked in code: $\max |P(S \mid D,H,W) - P(S \mid D,H)| < 10^{-16}$.

Listing all $2^5 = 32$ states gives a total probability of 1 (to within 2×10^{-16}) and the exact answers in Table 1. Two more checks matter. First, adding up the right states returns $P(H{=}1) = 0.30$ and $P(F{=}1) = 0.08$, and $P(H{=}1,F{=}1) = P(H{=}1)P(F{=}1)$, so the two causes really are independent before any evidence. Second, and more useful: I purposely swapped the H and F columns in the D table to plant a bug. The total *still* added to 1, so checking the total alone would have missed it — but q_1 changed from 0.2315 to 0.2044, so checking the answers caught it. Getting the total right is necessary but nowhere near enough. `A1_exact_reference_values.csv`

Table 1: Exact answers found by listing all 32 states. The last column shows why rare evidence makes rejection sampling slow.

Query	Meaning	Exact answer	P(evidence)	Samples kept per 1,000
q_1	$P(F{=}1 \mid S{=}1)$	0.231504	0.26816	268
q_2	$P(F{=}1 \mid S{=}1,W{=}1)$	0.293294	0.15932	159
q_3	$P(F{=}1 \mid S{=}1,W{=}1,H{=}1)$	0.168675	0.10956	110

A2. Forward sampling and rejection sampling

To draw a yes/no variable X with $P(X{=}1 \mid \text{parents}) = p$, pick a random number u evenly between 0 and 1 and set $x = 1$ if $u < p$. This works because u is spread evenly, so the chance of landing below p is exactly p: picture the line from 0 to 1 cut at p, where the two piece lengths *are* the two probabilities. I must pick the right table row first using the parent values already drawn, which is why the loop follows (H,F,D,S,W) — parents always come first, so the finished samples are genuine draws from the exact joint.

Rejection sampling throws away every sample that disagrees with the evidence and reports the share of survivors with $F{=}1$. This is correct because "restrict to the cases where the evidence happened" and "condition on the evidence" mean the same thing; the price is that only a fraction $P(e)$ survive. I generate one sample set per seed and reuse it for all three queries, at $N \in \{10^3, 10^4, 10^5\}$ with five seeds each. Survival rates match Table 1 well (e.g. 0.1590 against 0.15932 for q_2 at $N{=}10^5$). No answer was undefined across the 45 runs — the fewest survivors was 107, for q_3 at $N{=}10^3$ — but the code returns `NaN` for a $0/0$ case rather than inventing a number. `A2_rejection_per_run.csv`

A3. Likelihood weighting

I implemented likelihood weighting only, since the brief warns against adding Gibbs sampling just to write more code. The evidence variables are *forced* to their observed values, the rest are drawn as usual in parent-first order, and each sample gets a weight $w = \prod_{E_i \in \mathscr{E}} P(E_i{=}e_i \mid \mathrm{pa}(E_i))$, giving the weighted average $\hat{P}(F{=}1 \mid e) = \sum_{k:F^{(k)}=1} w^{(k)} / \sum_k w^{(k)}$.

Why the weights are needed. Forcing the evidence means drawing from the wrong distribution Q, not the posterior. Since $P(x_{-\mathscr{E}} \mid e) = Q(x_{-\mathscr{E}})\,w(x_{-\mathscr{E}})/P(e)$, the weight is exactly the correction between the two: it gives $\mathbb{E}_Q[w \mathbb{1}(F{=}1)] = P(F{=}1, e)$ and $\mathbb{E}_Q[w] = P(e)$, so dividing one by the other gives the answer. Counting *without* weights just returns the prior — I measured 0.0800 against the true $q_2 = 0.2933$ — because forcing a variable *below* F cannot change how F was drawn. The weights are the only thing carrying evidence back up the arrows. Dividing must use $\sum_k w^{(k)}$, which estimates $P(e)$ (average weight 0.1600 against the true 0.159316); dividing by N would give $P(F{=}1, e)$ instead. Note H in q_3 sits at the top of the graph, so it only contributes the fixed 0.30 — only evidence with parents makes the weights vary. `A3_likelihood_weighting_per_run.csv`

A4. Comparing the two methods

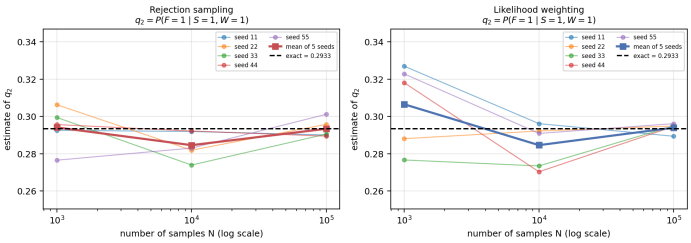


Table 2: Average ± spread over five seeds.

	N	Rejection	Lik. weighting
q_1	10^3	.238±.014	.251±.040
	10^4	.228±.007	.227±.009
	10^5	.231±.003	.231±.002
q_2	10^3	.294±.011	.307±.023
	10^4	.285±.008	.285±.012
	10^5	.293±.005	.294±.003
q_3	10^3	.170±.019	.173±.016
	10^4	.161±.018	.167±.004
	10^5	.169±.005	.169±.001

Figure 1: How q_2 settles down as N grows (five seeds; the dashed line is the exact answer). Both methods head to the right value, and the spread shrinks in step with $1/\sqrt{N}$. `A4_convergence_q2.png`, `A4_summary_table.csv`

What each new piece of evidence does. Start from the prior $P(F{=}1) = 0.08$. A "dry" reading lifts this to $q_1 = 0.2315$, about 2.9 times the prior, but still low: the sensor is only 90% accurate, and dry soil has a likelier rival cause, since hot weather starts at 0.30 against the fault's 0.08. Adding wilting raises it to $q_2 = 0.2933$, because wilting is a *second and separate* sign of dry soil — $S \perp W \mid D, H$ means the two are separate measurements of the same hidden thing, so it backs the sensor up instead of repeating it. Adding hot weather pushes the answer back *down* to $q_3 = 0.1687$. This is **explaining away**: D has two parents, so confirming one good rival cause makes the other less necessary. In plain terms, a dry-soil alert on a hot day is much weaker evidence of a fault than the same alert on a cool day. These are probabilities under the model we were given, not proof about a real greenhouse. `A4_evidence_sequence.csv`

Accuracy against effort. Rejection sampling keeps only a fraction $P(e)$ of its samples, so every extra piece of evidence shrinks what is left: at $N{=}10^5$ it keeps about 26,800 samples for q_1 but only 10,900 for q_3, while likelihood weighting keeps all N. With the five seeds the brief asks for, this is *not* reliably visible — rejection sampling wins a few rows in Table 2 — so I ran an extra check with 400 seeds. Likelihood weighting then has the smaller spread and error in **9 out of 9** (query, N) combinations, by about 1.04–1.26 times for q_1 and q_2 and a steady 2.05 times for q_3. Five seeds are simply not enough to rank two methods this close. On whether a few weights dominate, the effective sample size [9] $\mathrm{ESS} = (\sum w)^2 / \sum w^2$ divided by N gives 0.40, 0.22 and 0.46: the weight takes only a few values here and the largest 1% hold under 6% of the total, so no small group takes over — true for this small network with mild evidence, not in general. One fair-comparison note: rejection sampling makes one sample set shared by all three queries, while likelihood weighting reruns per evidence set, so I time them separately rather than pretending a "sample" means the same work in both. `A4_variance_check_400seeds.csv`, `A4_timing.csv`

Why more samples do not always help. Each estimate is itself random. A bigger N shrinks how much it *varies*, so the error falls on average and the seeds bunch closer — but a single run can get lucky at small N. For q_2 with rejection sampling, seeds 11 and 33 got *worse* from $N{=}10^3$ to 10^4, and seeds 11 and 44 from 10^4

to 10^5. Only averaged results say anything about a method; one run does not. That is exactly why five seeds are required.

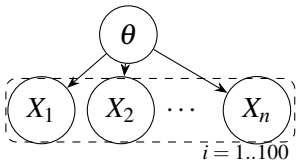
Why forcing later evidence is wrong. Forward sampling draws each variable from $P(x_i \mid \mathrm{pa}_i)$, so influence only flows parent to child; overwriting an observed child afterwards cannot change how its parents were drawn. I tried it: forcing $S{=}1, W{=}1$ across 200,000 samples gave 0.0799 — the prior — instead of $q_2 = 0.2933$. Diagnosis runs *against* the arrows, so a fix is compulsory: throw away samples that disagree (rejection), or reweight by how likely the evidence was.

Part B: learning with a Beta prior

B1. Setting up the problem

Each grasp either works or it does not, so the Bernoulli distribution fits: it is the only distribution on $\{0,1\}$ and has one parameter, $P(X_i{=}x \mid \theta) = \theta^x(1-\theta)^{1-x}$. For n independent attempts with s successes the likelihood is $\theta^s(1-\theta)^{n-s}$, depending on the data only through s and n.

The parameter θ is not something we observe. It is an unknown *continuous* number between 0 and 1, so describing our uncertainty needs a curve (a density), not one value. The Beta family lives exactly on the interval 0 to 1 — the right range for a probability. The key convenience is **conjugacy**: the Beta formula $\theta^{\alpha-1}(1-\theta)^{\beta-1}$ has the same shape as the Bernoulli likelihood, so multiplying them gives another Beta, and updating becomes arithmetic on two numbers. Note α and β are *prior settings, not data*: they act like $\alpha-1$ pretend successes and $\beta-1$ pretend failures, so $\mathrm{Beta}(2,3)$ is a slightly pessimistic starting belief (average 0.4) worth about three real attempts.



θ is the one shared parent of every observation, so $f(\theta, x_1..x_n) = f(\theta)\prod_i P(x_i \mid \theta)$ and $X_i \perp X_j \mid \theta$: once θ is fixed, knowing attempt 3 succeeded says nothing about attempt 4. But the attempts are *not* independent before we know θ. By simulation, under the prior $P(X_2{=}1 \mid X_1{=}1) = 0.4283$ while $P(X_2{=}1 \mid X_1{=}0) = 0.3335$; hold θ fixed and both become 0.55. This shared-parent link is what *makes learning possible* — if attempts were independent from the start, data could never teach us about θ. Note too that a density is not a probability: $f(0.8)$ can exceed 1 while $P(\theta{=}0.8) = 0$ exactly, which is why B3 asks for an area, not a height.

B2. Updating the posterior

Putting the Bernoulli likelihood and Beta prior into Bayes' rule, and sweeping every factor that does not contain θ into the constant:

$$f(\theta \mid \mathscr{D}) \propto \underbrace{\theta^s(1-\theta)^{n-s}}_{\text{likelihood}} \cdot \underbrace{\theta^{\alpha-1}(1-\theta)^{\beta-1}}_{\text{prior}} = \theta^{(\alpha+s)-1}(1-\theta)^{(\beta+n-s)-1} \implies \theta \mid \mathscr{D} \sim \mathrm{Beta}(\alpha+s,\, \beta+n-s).$$

So updating means: add successes to α, failures to β. Updating batch by batch and updating once with the totals *must* agree, because the likelihood multiplies across batches and multiplication ignores order. Both give $\mathrm{Beta}(2{+}85,\, 3{+}15) = \mathbf{Beta(87, 18)}$, checked by assertion. I also compared the shortcut against Bayes' rule computed numerically on a 200,001-point grid (largest difference about 10^{-9}) and confirmed all six batch orders agree. `B2_sequential_updates.csv`

Table 3: Running estimates. All three move together as the data (100 attempts) swamps the prior (worth about 3 attempts). `B2_point_estimates.csv`

Stage	Posterior	Total n	Total s	ML s/n	Posterior mean	MAP
Prior	$\mathrm{Beta}(2,3)$	0	0	—	0.4000	0.3333
After batch 1	$\mathrm{Beta}(8,7)$	10	6	0.6000	0.5333	0.5385
After batch 2	$\mathrm{Beta}(24,11)$	30	22	0.7333	0.6857	0.6970
After batch 3	$\mathrm{Beta}(87,18)$	100	85	0.8500	**0.8286**	0.8350

The distance between the posterior mean and the ML estimate shrinks from 0.067 after batch 1 to 0.021 after batch 3 — the prior pulls hardest when data are scarce. MAP sits between the two because this posterior leans left, so its peak is above its average.

B3. Uncertainty and the deployment decision

Three different quantities, all from Beta$(87,18)$:

(1) Prediction for the next attempt. We do not know θ, so instead of plugging in a guess we average over every value it might take: $P(X_{n+1}{=}1 \mid \mathscr{D}) = \int_0^1 \theta\, f(\theta \mid \mathscr{D})\, d\theta = \mathbb{E}[\theta \mid \mathscr{D}] = \mathbf{0.8286}$. The *number* matches the posterior mean, but the ideas differ — one is about a future observation, the other summarises a curve — and they agree only because averaging is a straight-line operation. For anything curved they part ways: the chance the next *two* attempts both succeed is $\mathbb{E}[\theta^2] = 0.6879$, above $(\mathbb{E}[\theta])^2 = 0.6865$, so plugging in the average understates it.

(2) 95% credible interval: $[\mathbf{0.7512}, \mathbf{0.8941}]$, checked by confirming the cumulative probability is 0.025 at the bottom and 0.975 at the top.

(3) Probability above the threshold: $P(\theta > 0.8 \mid \mathscr{D}) = \mathbf{0.7883}$, checked two ways — numerical integration agrees with SciPy [3] to eight decimal places, and 200,000 random draws from the posterior (seed `20260910`) give 0.78688, within half a standard error.

The decision: the rule is NOT met. It needs $P(\theta > 0.8 \mid \mathscr{D}) \geq 0.95$ and we have 0.7883, short by 0.1617; at the same success rate, roughly 400–500 more attempts would be needed to pass.

Why a good point estimate is not enough. Both the ML estimate (0.8500) and the posterior mean (0.8286) are *above* 0.8, and the rule still fails. A single number throws away how wide the posterior is: after 100 attempts the curve still has real width (spread 0.0366) and 21.2% of it sits below 0.8. The interval agrees — its lower end, 0.7512, is under the threshold, so $\theta < 0.8$ is not ruled out. The rule is written about the whole curve on purpose, because the cost of a bad gripper is paid when θ really is low, not when the average looks fine. To pass, the posterior must be *packed* above 0.8, not merely centred above it.

Why the height of the curve at 0.8 is not this probability. $f(0.8) = 7.26$ is a density — a rate of probability per unit of θ — so it cannot be compared with 0.95; this curve even peaks at 10.96, far above 1, and any single exact value has probability zero for a continuous quantity. What the rule needs is the *area* right of 0.8. The notebook shows the link: the probability of landing within ± 0.0005 of 0.8 is 0.0072565, and the height times that width is 0.0072556 — precisely what a density means. `B3_decision.json`, `B3_mc_verification.png`

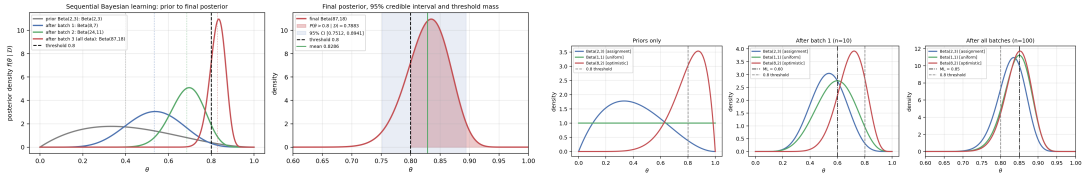


Figure 2: Left: learning batch by batch, then the final posterior with its 95% interval and the shaded area $P(\theta > 0.8 \mid \mathscr{D}) = 0.7883$ — clearly short of 0.95 even though the average is above 0.8. **Right:** three different priors after 0, 10 and 100 attempts; they end up almost on top of each other once enough data arrives.

B4. How much does the prior matter?

Table 4: Prior sensitivity. After 10 attempts the three priors disagree a lot; after 100 they nearly agree. `B4_prior_sensitivity.csv`

	After batch 1 (n=10, ML = 0.600)			After all batches (n=100, ML = 0.850)			
Prior	Mean	MAP	$P(\theta{>}0.8)$	Mean	MAP	$P(\theta{>}0.8)$	Rule met?
Beta$(2,3)$ assignment	0.5333	0.5385	0.0116	0.8286	0.8350	0.7883	No
Beta$(1,1)$ flat	0.5833	0.6000	0.0504	0.8431	0.8500	0.8811	No
Beta$(8,2)$ optimistic	0.7000	0.7222	0.1631	0.8455	0.8519	0.9009	No

After batch 1 the priors are worth 5, 2 and 10 pretend attempts against only 10 real ones, so the means spread from 0.533 to 0.700 and the threshold probabilities differ by a factor of 14 (0.012 to 0.163). After 100 attempts the data takes over: means bunch into 0.829–0.846, a spread of only 0.017, all close to ML = 0.850. The threshold probability stays more prior-sensitive than the mean (0.788–0.901), because it depends on the thin *tail* rather than the middle — worth knowing, since a rule written as a tail probability is shakier under a change of prior than one written as an average. The rule fails under all three priors, so that conclusion is safe; had it flipped, the honest reading would be that the data are too weak to decide.

ML and MAP with a flat prior. With Beta$(1,1)$ the prior is a flat line, so it contributes nothing that varies with θ, and maximising the posterior *is* maximising the likelihood: $\hat{\theta}_{\text{MAP}} = \frac{(1+s)-1}{(1+s)+(1+n-s)-2} = \frac{s}{n} = \hat{\theta}_{\text{ML}}$.

Confirmed in code at both stages (0.600 and 0.850).

Why that still does not make Bayes the same as ML. They agree only about the single highest point of the curve. The posterior *mean* with a flat prior is $(1+s)/(2+n) = 0.8431$, not 0.850, and that mean is the prediction for the next attempt — so even with a flat prior the Bayesian *prediction* differs from ML. More importantly, ML returns one number while the posterior returns a whole curve, and both the credible interval and $P(\theta > 0.8 \mid \mathscr{D})$ come from that curve; ML cannot produce what the rule asks for. The gap matters most when stakes are highest: with 5 successes out of 5, ML and MAP both say $\hat{\theta} = 1.0$ — claiming the next attempt *cannot* fail — while the flat-prior prediction gives 0.857 with interval $[0.541, 0.996]$. Picking a best value and carrying a distribution are different jobs, and only the second passes uncertainty into the decision.

Part C (option C2): drawing animals with flow matching

C1. The method and my two questions

Aim and data. A model that generates 28×28 grey line drawings on request: give it a category $y \in \{\text{cat}, \text{dog}, \text{fish}\}$ and random noise, and it produces a new drawing of that animal. Data: Google Quick, Draw! bitmap files (CC BY 4.0, Google Creative Lab) [2], 4,000 original drawings per category.

Flow matching in one idea. Take a noise image x_0 and a real drawing x_1 and slide in a straight line from one to the other. At time t you are at

$$x_t = (1-t)x_0 + t\,x_1, \qquad \text{moving at speed} \qquad u_t = \frac{dx_t}{dt} = x_1 - x_0, \qquad t \sim \mathscr{U}[0,1].$$

Because the path is straight the speed is the *same everywhere*, and it is just the difference between the two ends. So I train a network $v_\theta(t, x_t, y)$ to predict it with ordinary squared error: $\mathscr{L}_{\text{CFM}}(\theta) = \mathbb{E}\left\| v_\theta(t,x_t,y) - (x_1 - x_0)\right\|^2$. Nothing is simulated during training — the correct answer is available directly. To generate, I start from fresh noise and follow the predicted speeds from t=0 to t=1.

Training (correct answer known in advance): $x_0 \sim \mathscr{N}(0,I),\ x_1 \sim p_{\text{data}},\ t \sim \mathscr{U}[0,1] \ \rightarrow\ x_t = (1-t)x_0 + t x_1 \ \rightarrow\ v_\theta(t,x_t,y)$ squar
Generating (follow the speeds forward): $x_0 \sim \mathscr{N}(0,I) \ \rightarrow\ \left[x \leftarrow x + \Delta t \cdot v_\theta(t,x,y)\right] \times n_{\text{steps}},\ t: 0 \rightarrow 1 \ \rightarrow\ x(1) = \text{drawing}$

One worry: the same halfway image x_t could have come from many noise-and-drawing pairs, so which speed is "correct"? Squared error always settles on the *average* answer, so the network learns $\mathbb{E}[x_1 - x_0 \mid x_t, y]$ — and flow matching's key result [4,5] is that following those average speeds turns noise into the data distribution exactly. The network never guesses which pair it came from; it learns the typical direction of travel, which is all generation needs.

How this links to the unit. The same three ingredients as Parts A and B. *Known structure:* the noise distribution and the straight-line path, both chosen by me. *Unknown part learned from data:* the speed field — which, like the quantities in A and B, is an average. *Generating by sampling:* start from fresh noise and follow the path. The link to A4 is precise: the straight line is a *known* way to corrupt a drawing into noise, but knowing that forward direction does not give me the way back, because the average speed depends on what real drawings look like. That is the same trap as forcing evidence in a forward sampler and expecting the posterior.

Q1 (a training setting). Does training longer make drawings more recognisable, and does it cost variety? *Prediction:* a big jump from epoch 5 to 15, then flat; loss keeps falling; variety unchanged. **Q2 (a generation setting).** How few steps can I get away with, and does the more careful midpoint method beat plain Euler once network calls are counted fairly? *Prediction:* few steps needed, levelling off by 10–25, and at equal network calls midpoint gains little because straight paths barely bend.

C2. What I built, and three key pieces of code

Table 5: Baseline setup. Trained completely from scratch — no pretrained weights, nothing frozen. Implemented in PyTorch [8]; the reference implementation [6] was consulted for the 2-D starting point. `C_reproduction_record.json`

Data	4,000 drawings/class × 3	Network	`VelocityUNet` 28→14→7, ch. 64→128
Split	9,600 train / 2,400 held out	Parameters	1,838,081, all trainable
Preprocessing	$[0,255] \rightarrow [-1,1]$, no resize	Conditioning	time + class, fed into every block
Augmentation	random flip, training set only	Optimiser	AdamW, lr 2×10^{-4}, cosine decay
Filtering	ink between 5% and 55% (8/20,700 cut)	Batch / epochs	128 / 60, gradient clip 1.0
Seeds	split+train `20260910`; 32 for generating	Hardware / time	RTX 3060 (12 GB), **6.4 min**

(a) Building training examples — `flow_matching_batch` in `flow_model.py`, used inside `train()`. *In:* clean drawings `x1`, shape $(B, 1, 28, 28)$. *Out:* `(x_t, t, target)`. It draws noise `x0` of the same shape and one random time per image, reshaped so the same t applies to every pixel of that image, forms `x_t = (1-t)*x0 + t*x1`, and returns `x1 - x0` as the target. Every example uses fresh noise and time, so each drawing is seen at many noise levels.

(b) Loss and weight updates — the inner loop of `train()` in `train_flow.py`. *In:* a batch `(x1, y)`, the network, the optimiser. *Out:* one update step. It applies the flip augmentation (training set only), builds the three pieces above, then computes `loss = mean((model(t,x_t,y) - target)**2)`. That single line *is* the flow-matching loss, with the averaging over t and noise done by the random batch, followed by `zero_grad`→`backward`→clip→`step`. The whole generative model comes down to a regression. `eval_loss` uses a *fixed* set of t values so checkpoints compare fairly instead of differing by luck.

(c) Generating — `sample_ode` in `flow_model.py`. *In:* the network, how many images, the labels, `n_steps`, `method`, a `seed`. *Out:* the images, the **number of network calls (NFE)**, optionally the path. It seeds a generator, draws x_0 — *the only random thing in the process* — and steps t=0 to 1. Euler uses one call per step, midpoint two. I return the call count on purpose: comparing at the same *number of steps* would quietly give midpoint double the computing time.

Checks before the long run. I confirmed the shapes line up, that *every* weight gets a gradient, and specifically that the class embedding does — if that connection breaks the loss still falls nicely while every class produces the same picture, failing with no error message. I also checked the straight line starts and ends where it should (differences around 10^{-7}).

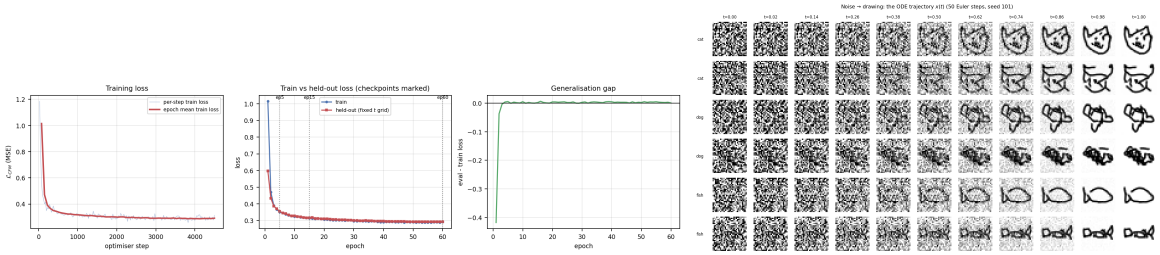


Figure 3: Left: training and held-out loss, with checkpoints marked. Held-out loss never rises more than 0.006 above training loss, so there is no overfitting at this size. **Right:** the journey from noise to drawing. The rough shape is set by about t=0.3–0.5 and the later steps just sharpen the lines — what you would expect if the paths are nearly straight.

C3–C4. Controlled comparisons and scoring

The protocol. I fixed a list of **32 seeds** once and reused it in *every* comparison. Each seed makes one drawing per class, so every setup gives **96 drawings** (32 per class, against the required minimum of 16 and 4). I went above the minimum on purpose: a first run with 8 seeds gave an identical 0.75 for all three checkpoints, the score simply being too coarse to tell them apart. Only one thing changes at a time. **The quality score** is *recognisability*: the share of a setup's 96 drawings that a separate small CNN files under the class I *asked for*. That CNN is trained *only on real held-out drawings* the generator never saw, and scores **0.8375** on them — so 0.8375, not 1.0, is the realistic maximum.

Table 6: Both comparisons. The margin of error is ±0.041 throughout (n=96), so any gap under about 0.09 means nothing. `C_all_results.csv`

	Setup	Held-out loss	Recog. (±0.041)	Confidence	Variety	NFE/img	ms/img
training	epoch 5	0.3557	0.781	0.637	22.6	50	16.9
	epoch 15	0.3164	0.771	0.672	22.1	50	16.9
	epoch 60 (baseline)	0.2922	**0.802**	**0.704**	22.5	50	16.9
	epoch 60, *quarter* of the data	0.3297	0.760	—	22.1	50	16.9
generating	Euler 2 steps	—	0.781	0.630	**14.9**	2	0.78
	Euler 5 steps	—	0.792	0.703	20.6	5	1.78
	Euler 10 steps	—	0.792	0.706	21.7	10	3.4
	Euler 50 steps	—	0.802	0.704	22.5	50	16.9
	Euler 100 steps	—	0.792	0.702	22.6	100	33.7
	Midpoint 5 steps	—	0.802	0.707	22.7	10	3.5



Figure 4: Matched grids with the same seeds down each column, showing the first 12 of 32 seeds, failures included. **Left (Q1):** only the checkpoint changes. The drawings clearly get better by eye — epoch-5 cats are scribbly, epoch-60 cats have clean ears and faces — even though the recognisability *score* hardly moved. **Right (Q2):** only the number of steps changes. The 2-step row is visibly faint and washed out, while 10 and 50 steps look almost the same. All 96 drawings for all 14 setups, including raw `.npz` arrays, are in `results/partC/sample_grids/`.

C5. What the results mean

Q1: my prediction was wrong, and how it was wrong is the most useful finding here. Held-out loss dropped steadily by 18% (0.356 → 0.292) and average confidence rose steadily (0.637 → 0.704), but *recognisability barely moved*: 0.781 → 0.771 → 0.802, half the margin of error. I cannot claim a real gain from training 12 times longer. Three things follow. *First*, this is a clear numerical demonstration that **training loss does not measure picture quality**: the loss averages speed errors over all times t, and most of the leftover improvement happens at noise levels that barely affect where the path lands. *Second*, confidence was the more sensitive measurement — it rose steadily while yes/no scoring throws that information away, so longer training sharpens drawings that were *already* classified correctly. Reporting only the score would have hidden a real effect; reporting it *as if* it showed one would have invented an effect. *Third*, the task levels off early: by epoch 5 (32 seconds) drawings already score 0.781 against a maximum of 0.8375. Figure 4 shows the drawings really *do* get better, so the honest conclusion is that my *measurement* was too blunt here, not that training does nothing. Per-class numbers locate the difficulty: `fish` scores 0.94–0.97 at every checkpoint while `dog` scores 0.53–0.66, matching the classifier's own limits on real drawings (fish 0.916, cat 0.821, dog 0.781) — dogs are hardest for scorer and generator alike, so this is the data, not broken conditioning. The *data-size* comparison does move things: quartering the training set at identical training time pushes held-out loss 0.292 → 0.330 and recognisability 0.802 → 0.760, so at this network size the limit is data, not training time.

Q2: both halves of my prediction held, and the levelling-off is even more extreme than expected. Recognisability is flat from 2 to 100 Euler steps, all within one margin of error. What *does* change at 2 steps is **variety**, collapsing from about 22.5 to **14.9** — a 34% drop, far bigger than any wobble elsewhere — with confidence also falling to 0.630. So the 2-step failure is not "wrong animal" but "not finished": drawings land roughly in the right place yet come out faint and too similar, exactly as Figure 4 (right) shows. Past about 5 steps, extra steps buy nothing measurable while cost climbs linearly — 0.78 ms to 33.7 ms per image from 2 to 100 steps, 43 times the cost for no gain. This is flow matching's practical advantage over diffusion sampling [7]: straight-line training gives nearly straight paths, so a handful of steps is enough where diffusion needs hundreds. Comparing solvers **at equal network calls**, the differences are $+0.010$, -0.010 and 0.000, each far below the ± 0.058 margin of error, so none is distinguishable. That fits the geometry: a more careful solver only pays off when the path bends, and these are trained nearly straight. Midpoint's only visible edge is at the cheapest setting, recovering variety 23.7 on 4 network calls against Euler's 14.9 on 2. `C_comparison2_matched_nfe.csv`

Why the output is still random when the solver is not. Both solvers are fixed recipes: pin down x_0 and the output is pinned down too, which is what makes my 32-seed list repeatable. All randomness enters through the *starting noise* x_0; the path just carries the noise distribution onto the drawing distribution. Variety comes from where you start, not from anything added along the way — the real difference from a diffusion sampler [7], which adds new randomness every step. I checked this rather than assuming it, and the check caught something: with default settings two runs of the *same* seed differed by about 7×10^{-4}, because the GPU library may pick different convolution algorithms each time. The maths was never at fault (x_0 was identical), but "same seed, same picture" held only to plotting accuracy until I pinned the GPU settings

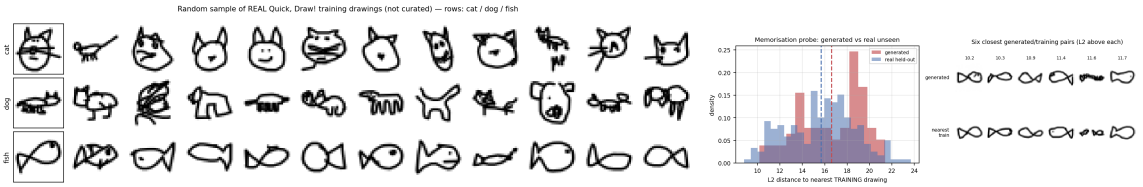


Figure 5: Left: a random (not hand-picked) sample of the real *training* drawings, looked at before training. Several are unclear even to a person — they are 20-second public doodles — and that is what sets the 0.8375 ceiling rather than 1.0. **Right:** the copying check. Generated drawings sit at average distance 16.62 from their closest training drawing versus 15.69 for real held-out drawings, and even the six closest pairs share only a rough layout, not strokes.

(`enable_deterministic()`). Now same-seed runs match exactly, and the notebook checks it.

Copying. The average distance from each of the 96 generated drawings to its closest *training* drawing is **16.62**, against **15.69** for 200 real *held-out* drawings measured the same way — so generated drawings are actually *further* from the training set than genuinely unseen real drawings, which argues against copying. The six closest pairs share only a rough layout (a body outline, a tail pointing the same way), not the strokes. Looking similar is not proof: with 3,200 simple doodles per class, any believable new cat *has* to resemble some training cat, so the meaningful comparison is the spread of distances, not a single pair. `C_memorisation_distances.csv`

Limits of my score, and of the study. (i) The realistic maximum is 0.8375, not 1.0, because the doodles are often unclear even to a person; epoch 60 reaches 96% of it. (ii) The scoring CNN has biases of its own, so a good score is necessary but not sufficient for "a person would call this a cat". (iii) **It measures whether the class is right, not whether the drawing is good, and this caught me out**: the 2-step drawings scored 0.781, indistinguishable from 50 steps, despite being obviously worse and losing 34% of their variety — nearly blind to the biggest quality difference in the study. (iv) With 96 drawings the margin of error is 0.041, so gaps under about 0.09 cannot be resolved; only two effects beat the noise (the variety collapse at 2 steps, the loss worsening on a quarter of the data) and the rest are genuine null results. Finding smaller effects needs several hundred drawings per setup — affordable at 17 ms each, and the first thing I would change. (v) Straight-line distance between images is weak: fine for near-duplicates, blind to a copy shifted a few pixels. (vi) 28×28 line drawings are the easy end of image generation, and nothing here shows the method scales to photographs. I did not compute FID or run a human study — both need far more samples or a pretrained feature extractor.

Conclusion: inference → learning → generating

Part A used a *fully known* model, where the only real difficulty was handling evidence properly: overwriting later evidence gave back the prior (0.0799 against the true $q_2 = 0.2933$), and rejection sampling and likelihood weighting are the two valid repairs. Part B kept the structure known but made *one number* unknown, and learning a full curve over it changed the decision: ML $= 0.850$ sits above the threshold, yet $P(\theta > 0.8 \mid \mathscr{D}) = 0.788$ fails the 0.95 rule, because a single number throws away the very width the rule is about. Part C made the unknown a whole *function*: I chose the noise distribution and the path, and the network learned the speed field — like the quantities in A and B, just an average. Generating is then sampling in the same sense: draw from something simple and push it through what was learned. The thread through all three is that a generative model is not a different *kind* of thing from a Bayesian network — it is the same split-up, learn, sample pattern with a network in place of a probability table, and the same warning applies at every size: knowing the forward direction never gives you the reverse for free. Part C added the lesson the brief points at: an 18% better training loss produced no measurable change in drawing quality, so the measurement you pick decides what you can conclude.

AI-use statement

I used Claude (Anthropic), model Opus 5, through the Claude Code CLI on 9–10 September 2026, for explanations of flow matching, code scaffolding for the network and sampler, notebook structure and debugging. All the maths, experiment design, checking and claims are my own responsibility, and every number reported came from code I ran on my own machine. Five interactions are written up in `AI_USE_LOG.md` with the prompt, the relevant reply, my check and what I decided. They include one independently verified correctness check (I re-derived Part A's exact answers with a separate brute-force program, agreeing to 10 decimal places, and added a swapped-table test the assistant had not suggested, showing the total-adds-to-1 check is weaker than it looks) and one prediction-versus-result comparison (the Euler/midpoint expectation above, which the data

contradicted). I also record one suggestion I **rejected**: the proposed ink-filter limits of 2% and 60% cut out *nothing* once I measured the real distribution, so I recalibrated to 5% and 55%. The log's table lists all 16 checks I ran. No result, runtime, citation or figure in this submission was made up.

References

1. COMP6010 teaching materials (2026): *Bayesian Networks I*; *Bayesian Networks 2 – Inference*; *Learning Probabilistic Models*; *Diffusion and Flow Matching Models*; continuous-probability notes; Practicals 1–4, 6 and 7. Primary source for all three parts.
2. Google Creative Lab. *The Quick, Draw! Dataset* (bitmap format), CC BY 4.0. github.com/googlecreativelab/quickdraw-dataset. Categories `cat`, `dog`, `fish` retrieved 10 Sept 2026; checksums in `C_data_manifest.json`.
3. SciPy documentation, `scipy.stats.beta` (v1.15.3) — Beta densities, cumulative functions, quantiles and sampling.
4. Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M. (2023). Flow Matching for Generative Modelling. *ICLR*. arXiv:2210.02747
5. Liu, X., Gong, C., Liu, Q. (2023). Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. *ICLR*. arXiv:2209.03003
6. Meta AI. *flow_matching* library and example notebooks. github.com/facebookresearch/flow_matching
7. Ho, J., Jain, A., Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *NeurIPS*. arXiv:2006.11239 — used for the comparison in §C5.
8. Paszke, A. et al. (2019). PyTorch (v2.10.0+cu128).
9. Kish, L. (1965). *Survey Sampling*. Wiley — effective sample size used in §A4.